Report

Buffer Overflow and Long Password DoS Attack

Suyash A Gunjal

Buffer Overflow

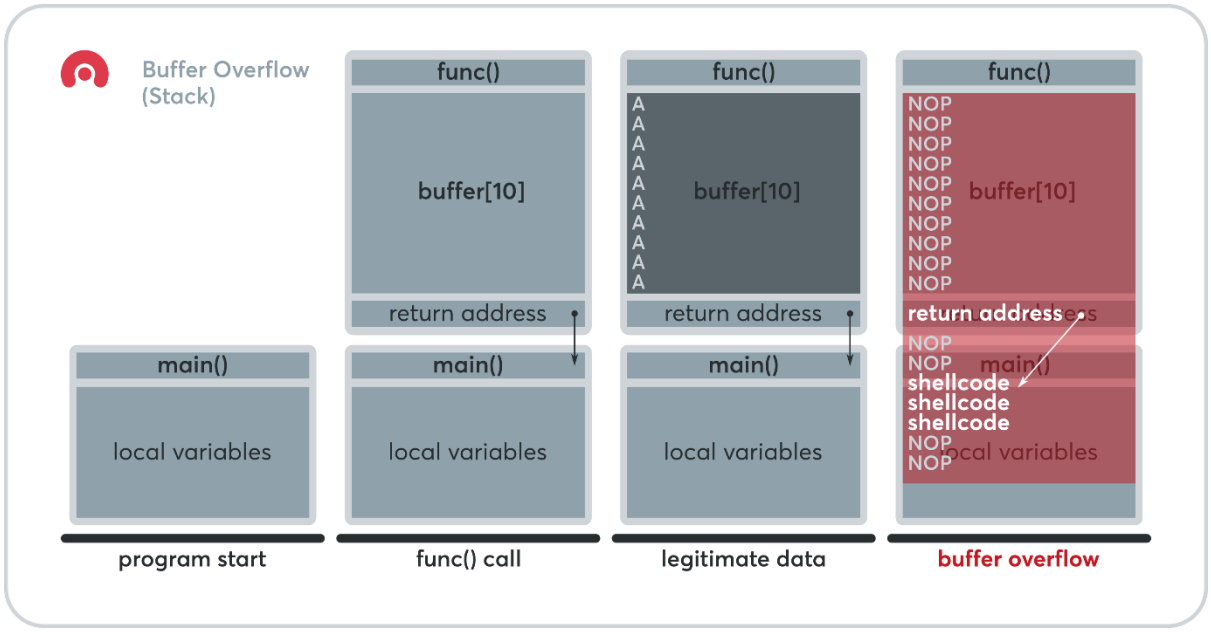
What is Buffer Overflow?

Buffer overflow is a software coding error or vulnerability that can be exploited by hackers to gain unauthorized access to corporate systems.

It is one of the best-known software security vulnerabilities yet remains common.

This is partly because buffer overflows can occur in various ways and the techniques used to prevent them are often error prone.

The software error focuses on buffers, which are sequential sections of computing memory that hold data temporarily as it is transferred between locations.



Why Do Buffer Overflows Happen?

They happen mainly due to unsafe input handling, especially in low-level languages like C and C++, where memory management is manual and not automatically checked.

Main causes:

1. No boundary checking on user input
2. Use of insecure/legacy functions (e.g., gets(), strcpy(), sprintf())
3. Mixing untrusted input with fixed memory allocations

How a Buffer Overflow Works (3 Points)

1. More data is written to a buffer than it can hold, causing extra data to spill into adjacent memory.
2. The overwritten memory may include return addresses or control structures, allowing attackers to change program execution.
3. Attackers can inject malicious code or redirect execution (e.g., to shellcode) to gain unauthorized control of the system or crash the application.

How to Prevent Buffer Overflow (3 Points)

1. **Use safe input handling and bounds checking**, e.g., strncpy() instead of strcpy() and validating input size.
2. **Enable modern protection mechanisms** such as ASLR, DEP/NX, stack canaries, compiler hardening (-fstack-protector).
3. **Use secure programming practices and updated languages/libraries**, avoid unsafe functions, keep software patched, and perform code reviews/testing (fuzzing).

Long Password DoS Attack

What is Long Password DoS Attack?

Long Passwords: In many systems, users are allowed to set their passwords. Some systems may not have a maximum password length limit, or the limit might be set very high.

As a result, users can create extremely long passwords, which may consist of many characters

Resource Consumption: The key to this type of DoS attack is that when a user with a very long password attempts to log in, the system or service must process and store that lengthy input.

Processing and storing long passwords can consume more server resources (such as CPU and memory) and can take longer to validate, especially if the system performs complex hashing or encryption on passwords.

How Long Password DoS Attack Works?

The attacker sends very large password input to the login, signup, or password-reset request (e.g., 50,000–1,000,000 characters).

The backend tries to **hash,** encode, or compare this long input using heavy algorithms like bcrypt, PBKDF2, scrypt, or Argon2.

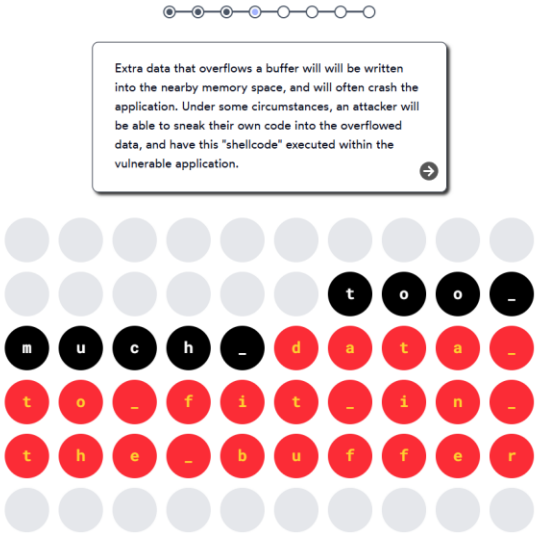
This causes high CPU usage, long processing time, thread blocking, and memory strain, resulting in slow response or full denial-of-service (DoS).

How to prevent Long Password DoS Attack?

1. Enforce strict server-side password length limits (e.g., max 64–128 characters) and reject oversized input before processing or hashing.

2. Apply rate-limiting, request throttling, and account lockout policies to prevent repeated abuse from the same user or IP address.

3. Validate password length before running expensive hashing operations and use timeouts or request-size filters at the application or WAF level.



References:

https://www.acunetix.com/blog/web-security-zone/what-is-buffer-overflow/

https://hacksplaining.com/lessons/ai-bias-and-unreliability/bias